

통합 씬 데이터 드리븐 아키텍처 설계

목적: 시나리오마다 별도의 씬을 만드는 대신, **하나의 통합 씬**에 데이터를 주입하여 내부 오브젝트를 상황에 맞게 배치/컨트롤하는 구조 설계

현재 상태: 씬 18개 존재 (Chuna_upper_Seat, Chuna_SCM_new, Chuna_Scalene_new, Chuna_Chest, Scenario_1~5 등)

목표: 하나의 "통합 TrainingScene" + ScenarioConfig 데이터로 모든 시나리오 처리

1. 씬마다 달라지는 요소 (= 데이터화해야 할 것)

구분	내용	예시
환자 자세/위치	앉아있음, 누워있음, 옆으로 누움	Seated, Supine, SideLying
시술 시나리오	어떤 근육에 대한 시술인지	상부승모근, 견갑거근, 사각근, 대흉근
환경 배치	침대 유무/위치, 카메라 위치, 골격모델	침대 ON/OFF, 카메라 각도
리소스 참조	애니메이션, 핸드데이터, 나레이션, 영상	CSV, AudioClip, VideoClip

2. 마스터 데이터: ScenarioConfig (ScriptableObject)

시나리오별 1개의 예셋. `Resources/ScenarioConfigs/` 폴더에 저장.

```
[CreateAssetMenu(fileName = "NewScenarioConfig", menuName = "GuideChuna/ScenarioConfig")]
public class ScenarioConfig : ScriptableObject
{
    [Header("=== 기본 정보 ===")]
    public string scenarioId;           // 예: "upper_trapezius"
    public string scenarioName;         // 예: "상부승모근"
    public string csvFileName;          // 예: "상부승모근" (Resources/Scenarios/ 하위)

    [Header("=== 환자 자세/위치 ===")]
    public PatientPose patientPose;     // enum: Seated / Supine / Prone / SideLying
    public Vector3 patientPosition;
    public Vector3 patientRotation;

    [Header("=== 침대 설정 ===")]
    public bool bedActive = true;
    public Vector3 bedPosition;
    public Vector3 bedRotation;

    [Header("=== 애니메이션 ===")]
    public RuntimeAnimatorController animatorController;

    [Header("=== 접촉 감지 기본값 ===")]
    public string defaultContactTarget; // Head / HeadAndShoulder / Chest
    public string pivotTarget;          // Neck / LeftShoulder / RightShoulder
    public string pivotPlaneAxis;        // X / Y / Z
    public bool invertAngle;

    [Header("=== 리소스 경로 ===")]
    public string handDataFolder;        // 예: "HandPoseData"
    public string narrationFolder;       // 예: "Narrations/Beginner"
    public VideoClip guideVideoClip;     // 가이드 영상 (또는 string 경로)

    [Header("=== 카메라/골격 모델 배치 ===")]
    public Vector3 cameraPosition;
    public Vector3 cameraRotation;
    public Vector3 skeletonModelPosition;
    public Vector3 skeletonModelRotation;

    [Header("=== UI 배치 ===")]
    public float uiForwardDistance = 1.5f;
}
```

```
[Header("=== 추가 프리팹 ===")]
public GameObject[] additionalPrefabs; // 시나리오별 특수 소품
}

public enum PatientPose
{
    Seated,
    Supine,
    Prone,
    SideLying
}
```

3. 전체 데이터 흐름도

3-1. 씬 전환 흐름 (로비 → 통합 씬)

[로비 씬] 사용자가 시나리오 카드 클릭

```
|  
|  
| ① 선택 정보를 PlayerPrefs에 저장  
|  
▼
```

```
|  
| PlayerPrefs (씬 간 데이터 전달)  
|  
| "SelectedMode"      = "학습" / "실습"  
| "SelectedDifficulty" = "Beginner" / "Intermediate" /...  
| "SelectedScenario"  = "upper_trapezius" ← ★ 핵심 키  
| "LOGIN_USERNAME"    = "홍길동"  
| "LOGIN_USERID"      = 42  
|
```

```
|  
| ② SceneLoader.LoadScene("TrainingScene")  
| (통합 씬 하나만 로드)  
▼
```

```
|  
| LoadingScene (비동기 로드)  
|
```

```
|  
| 통합 TrainingScene (하나의 씬)  
|  
| ③ ScenarioBootstrapper.Start()  
| |  
| | scenarioId = PlayerPrefs.GetString  
| | ("SelectedScenario")  
| |  
| ▼  
| ④ ScenarioConfig 로드  
| |  
| | config = Resources.Load<ScenarioConfig>  
| | ("ScenarioConfigs/" + scenarioId)  
| |  
| ▼  
| ScenarioConfig (ScriptableObject)
```

해당 시나리오의 설정 데이터 전부 담고 있음

⑤-A 환경 셋업 ⑤-B 환자 셋업 ⑤-C 시나리오 로드

3-2. 초기화 상세 (ScenarioBootstrapper가 config 기반으로 배치)

ScenarioConfig

```
|
|
|----- ㉠-A 환경 셋업 -----
|
|
|   | PatientPositionManager.ApplyFromConfig(config)
|   |   | 환자 위치/회전 설정
|   |   | 침대 위치/활성화 설정
|   |   |   └─ 골격 모델/카메라 위치 설정
|   |
|   |
|   |─ 추가 프리랩 Instantiate (config.additionalPrefabs)
|   |   (시나리오별 특수 소품이 있으면 동적 생성)
|   |
|   |─ ScenarioUIPositioner 설정 갱신
|   |   (UI 배치 거리 등)
|
|
|----- ㉠-B 환자 셋업 -----
|
|
|   |─ Animator Controller 교체
|   |   patient.runtimeAnimatorController
|   |       = config.animatorController
|   |
|   |─ 접촉 감지 기본 부위 설정
|   |   ChunaPathEvaluator.SetContactTarget
|   |       (config.defaultContactTarget)
|   |
|   |─ 피벗 포인트 설정
|   |   (Neck / LeftShoulder / RightShoulder)
|
|
|----- ㉠-C 시나리오 데이터 로드 -----
|
|
|   |─ ScenarioCSVLoader.LoadScenarios(config.csvFileName)
|   |   |
|   |   |   └─ Resources/Scenarios/{csvFileName}.csv
|   |   |
|   |   |   ▼
|   |   |   ScenarioData
|   |   |       └─ PhaseData[]
|   |   |           └─ StepData[]
|   |   |               └─ SubStepData[]
|   |   |                   └─ handTrackingFileName
```

```
| | | patientAnimationClip
| | | voiceInstruction
| | | contactTarget
| | | videoStartTime/EndTime
| |
| | └─ SceneManager.StartScenario(scenarioData)
|
└─ ㉔-D 난이도 적용 ───────────────────
    |
    └─ DifficultyManager.SetDifficulty
        (PlayerPrefs["SelectedDifficulty"])
        |─ 가이드 핸드 표시 여부
        |─ 나레이션 타임 결정
        └─ 평가 임계값 설정
```

3-3. 런타임 동작 흐름 (시나리오 실행 중)

ScenarioManager (Phase → Step → SubStep 순회)

|
| SubStep 시작될 때마다



ScenarioEventSystem.SubStepStarted(subStep)

|
| ↳ 나레이션 재생
| narrationFolder = config.narrationFolder
| clipName = subStep.voiceInstruction
| clip = Resources.Load(narrationFolder + "/" + clipName)
|
| ↳ 핸드데이터 로드 & 평가 시작
| csvPath = config.handDataFolder + "/" +
| subStep.handTrackingFileName
| ChunaPathEvaluatorBridge.LoadFromCSV(csvPath)
|
| ↳ 환자 애니메이션 설정
| animator.Play(subStep.patientAnimationClip)
| (AnimatorController는 이미 ㉟-B에서 교체됨)
|
| ↳ 가이드 영상 구간 재생
| GuideVideoController.PlaySegment
| (subStep.videoStartTime, subStep.videoEndTime)
|
| ↳ 접촉 감지 부위 갱신
| ChunaPathEvaluator.SetContactTarget
| (subStep.GetContactTarget())

3-4. 결과 & 복귀

시나리오 완료

|
| ↳ TrainingResultTracker.FinishTracking()
| → 결과 데이터 수집
|
| ↳ 결과 패널 / 퀴즈 표시
|
| ↳ SceneLoader.LoadScene("lobby")
| → 로비로 복귀 (다른 시나리오 선택 가능)

4. 데이터 저장 방식 비교

데이터 종류	저장 방식	현재 상태	통합 씬에서
시나리오 진행 정보	CSV (Resources)	이미 사용 중	그대로 유지
핸드 포즈 기준 데이터	CSV (Resources)	이미 사용 중	그대로 유지
나레이션 오디오	AudioClip (Resources)	이미 사용 중	그대로 유지
선택 모드/난이도	PlayerPrefs	이미 사용 중	그대로 유지
로그인 정보	PlayerPrefs	이미 사용 중	그대로 유지
환자 위치/자세	Inspector 하드코딩	씬마다 고정	ScenarioConfig
침대/카메라/환경 배치	Inspector 하드코딩	씬마다 고정	ScenarioConfig
Animator Controller	Inspector 할당	씬마다 고정	ScenarioConfig
접촉 감지 기본값	Inspector 할당	씬마다 고정	ScenarioConfig
추가 소품/프리팸	씬에 직접 배치	씬마다 고정	ScenarioConfig

핵심: Inspector에 하드코딩된 것들을 ScenarioConfig ScriptableObject로 빼내는 것

5. 새로 만들어야 할 핵심 컴포넌트

5-1. ScenarioConfig (ScriptableObject)

- 시나리오별 환경/환자/리소스 설정 데이터
- `Resources/ScenarioConfigs/` 에 에셋으로 저장
- Inspector에서 쉽게 편집 가능
- 위 섹션 2의 코드 참고

5-2. ScenarioBootstrapper (MonoBehaviour)

```
/// <summary>
/// 통합 씬 진입 시 가장 먼저 실행.
/// PlayerPrefs에서 scenarioId를 읽고, ScenarioConfig를 로드하여
/// 각 매니저에 config를 전달하여 초기화한다.
/// </summary>
public class ScenarioBootstrapper : MonoBehaviour
{
    [Header("=== 매니저 참조 ===")]
    [SerializeField] private PatientPositionManager positionManager;
    [SerializeField] private ScenarioManager scenarioManager;
    [SerializeField] private ScenarioCSVLoader csvLoader;
    [SerializeField] private ScenarioUIPositioner uiPositioner;

    [Header("=== 환자 참조 ===")]
    [SerializeField] private Animator patientAnimator;
    [SerializeField] private ChunaPathEvaluator pathEvaluator;

    [Header("=== 디버그 ===")]
    [SerializeField] private string debugOverrideScenarioId = "";

    private ScenarioConfig currentConfig;

    void Start()
    {
        string scenarioId = !string.IsNullOrEmpty(debugOverrideScenarioId)
            ? debugOverrideScenarioId
            : PlayerPrefs.GetString("SelectedScenario", "");

        if (string.IsNullOrEmpty(scenarioId))
        {
            Debug.LogError("[Bootstrapper] SelectedScenario가 설정되지 않았습니다!");
            return;
        }

        // ① ScenarioConfig 로드
        currentConfig = Resources.Load<ScenarioConfig>(
            $"ScenarioConfigs/{scenarioId}");

        if (currentConfig == null)
        {

```

```

        Debug.LogError($"[Bootstrapper] ScenarioConfig를 찾을 수 없습니다: {scenarioId}");
        return;
    }

    // ㉔ 환경 배치
    SetupEnvironment(currentConfig);

    // ㉕ 환자 셋업
    SetupPatient(currentConfig);

    // ㉖ 시나리오 CSV 로드 & 시작
    LoadAndStartScenario(currentConfig);
}

private void SetupEnvironment(SenarioConfig config) { /* ... */ }
private void SetupPatient(SenarioConfig config) { /* ... */ }
private void LoadAndStartScenario(SenarioConfig config) { /* ... */ }
}

```

5-3. EnvironmentConfigurator (MonoBehaviour) - 선택사항

- ScenarioConfig 받아서 환경 오브젝트 배치
- 침대 ON/OFF, 추가 소품 생성/파괴
- `PatientPositionManager.ApplyFromConfig()` 호출
- ScenarioBootstrapper에 통합하거나 별도 클래스로 분리 가능

6. 기존 코드 재활용 포인트

현재 프로젝트에 이미 존재하는 모듈화된 컴포넌트들:

기존 컴포넌트	역할	통합 씬에서 변경사항
ScenarioCSVLoader	CSV 파싱 → ScenarioData	변경 없음 - csvFileName만 외부에서 전달
ScenarioManager	Phase→Step→SubStep 진행	변경 없음 - 이미 데이터 기반 동작
PatientPositionManager	프리셋 기반 위치 설정	소폭 수정 - ApplyFromConfig() 메서드 추가
ScenarioUIPositioner	헤드셋 기준 UI 배치	소폭 수정 - forwardDistance 외부 주입
ScenarioConditionManager	조건 판정	변경 없음
ChunaPathEvaluatorBridge	핸드포즈 평가 연동	변경 없음
LoginStateStore	로그인 정보 관리	변경 없음
PrefsKeys	PlayerPrefs 키 상수	SelectedScenario 키 추가

결론: ScenarioConfig + ScenarioBootstrapper 두 개만 추가하면 기존 코드를 크게 건드리지 않고 통합 씬 전환이 가능

7. 구현 우선순위

1. ScenarioConfig ScriptableObject 생성 - 데이터 구조 정의
2. PrefsKeys에 "SelectedScenario" 추가 - 씬 간 데이터 전달 키
3. ScenarioBootstrapper 구현 - 통합 씬 초기화 로직
4. PatientPositionManager에 ApplyFromConfig() 추가 - config 기반 위치 적용
5. 통합 TrainingScene 구성 - 모든 공통 오브젝트 배치
6. 기존 씬에서 ScenarioConfig 에셋 추출 - 각 씬의 Inspector 값을 에셋으로 변환
7. 로비 씬에서 SelectedScenario 저장 로직 추가
8. 테스트 & 기존 씬 제거